INTERN TECHNICAL ASSIGNMENT

BUG MANAGEMENT SYSTEM

Assignment objective
Build a basic full-stack Bug Management System to demonstrate practical skills in REST API development, relational database operations, React frontend development, and frontend–backend integration.

1. Assignment Overview

Develop a simple **Bug Management System** using the following technologies:

- **Backend:** ASP.NET Core Web API (.NET)
- **Frontend:** React.js
- **ORM:** Entity Framework Core (EF Core)
- **Database:** PostgreSQL or SQL Server

The purpose of this assignment is to evaluate your understanding of REST API development, database operations, frontend development, and integration between the frontend and backend.

2. Functional Requirements

The application should allow users to manage bugs through basic CRUD operations. Users should be able to:

- Create a new bug.
- View a single bug by its ID.
- View all reported bugs.
- Update an existing bug.
- Delete a bug.

3. Bug Entity

Create a database entity named **Bug** with the following attributes:

Attribute	Data Type	Description
Id	int / Guid	Unique identifier (Primary Key)
BugName	string	Name or title of the bug
BugDescription	string	Detailed description of the bug
ReportDate	DateTime	Automatically assigned current date and time
Level	Enum	Severity level of the bug

Bug severity levels

The **Level** attribute should support the following values: **Critical, High, Medium,** and **Low**.

Important: The ReportDate must be automatically generated by the backend using the current server date and time when a new bug is created. The frontend should not provide this value.

4. Backend Requirements – ASP.NET Core Web API

Implement the following REST API endpoints:

Method	Endpoint	Description
POST	/api/bugs	Create a new bug
GET	/api/bugs/{id}	Retrieve a bug by ID
GET	/api/bugs	Retrieve all bugs
PUT	/api/bugs/{id}	Update an existing bug
DELETE	/api/bugs/{id}	Delete a bug

API requirements

- Use ASP.NET Core Web API.
- Use Entity Framework Core for database operations.
- Configure the database connection properly.
- Implement the required CRUD operations.
- Use appropriate HTTP status codes.
- Handle cases where a requested bug does not exist.
- Implement basic input validation.
- Return appropriate responses from the APIs.

5. Frontend Requirements – React

Develop a simple and user-friendly React application with the following features:

5.1 Bug Listing Page

- Display all reported bugs.
- Show Bug Name, Description, Report Date, and Severity Level.
- Provide Edit and Delete actions for each bug.
- Display an appropriate message when no bugs are available.

5.2 Create Bug

- Provide a form to create a new bug.
- Include fields for Bug Name, Bug Description, and Severity Level.
- Submit the form to the backend API.
- Refresh the bug list after successful creation.

5.3 Edit Bug

- Allow users to select an existing bug for editing.
- Load the existing bug details into the form.

- Allow users to update the bug name, description, and severity level.
- Save the changes through the update API.

5.4 Delete Bug

- Allow users to delete a bug.
- Display a confirmation before deleting.
- Refresh the bug list after successful deletion.

5.5 View Bug Details

- Allow users to view the details of an individual bug.

6. Technical Requirements

- Use a proper project structure for both frontend and backend.
- Follow basic coding conventions and naming standards.
- Use Entity Framework Core migrations to manage database changes.
- Keep API endpoints and frontend components organized.
- Use React functional components and hooks.
- Handle API errors appropriately.
- Do not hardcode database credentials in source code.
- Keep the frontend and backend in separate projects.

7. Expected Deliverables

Submit the following items:

- Complete backend source code.
- Complete frontend source code.
- Database migration files.
- **README.md** containing project setup instructions, required dependencies, database configuration instructions, instructions to run the backend and frontend, and available API endpoints.
- Screenshots of the working application.

8. Evaluation Criteria

The assignment will be evaluated based on the following criteria:

- Correct implementation of CRUD operations.
- Proper use of ASP.NET Core Web API.
- Effective use of Entity Framework Core.
- Correct database design and relationships (where applicable).
- React component structure and functionality.
- Frontend and backend integration.
- Input validation and error handling.
- Code readability and project organization.

9. Scope Restrictions

This is a basic CRUD assignment. The following features are **not required**:

- User authentication or authorization.
- Role management.
- File or image uploads.
- Email notifications.
- Advanced reporting or analytics.

Expected Outcome
A fully functional Bug Management System where users can create, view, update, and delete bugs through a React frontend connected to an ASP.NET Core Web API and a relational database.